

MEMORIA DINÁMICA EN C - BLOQUES CORREGIDOS

Pensó durante unos pocos segundos

Claro. Según tus apuntes, el apartado 10 es Memoria dinámica, dividido en:

Petición de memoria: malloc, free, calloc, sizeof, realloc, strdup

Bloques de memoria: memcpy, memset, memcmp

Voy por partes.

10. Memoria dinámica en C

En C tienes dos formas principales de guardar datos:

-----**código**-----

```
int x = 5;
int array[10];
```

-----**código**-----

Eso es memoria estática/local automática. El tamaño se conoce normalmente antes o durante la ejecución de la función, y la memoria se libera sola cuando sales del bloque.

Pero a veces necesitas memoria cuyo tamaño no conoces hasta que el programa está funcionando.

Ejemplo:

-----**código**-----

```
int n;
scanf("%d", &n);

int array[n]; // esto existe en C moderno, pero no siempre es lo ideal
```

-----**código**-----

Para eso existe la memoria dinámica.

La memoria dinámica se pide manualmente usando funciones como:

-----**código**-----

```
malloc
calloc
realloc
```

-----**código**-----

strdup

Y se libera manualmente con:

free

La idea central es:

-----**código**-----

```
int *p = malloc(...);
```

malloc devuelve una dirección de memoria. Esa dirección la guardas en un puntero.

-----**código**-----

1. sizeof

Antes de malloc, hay que entender sizeof.

sizeof devuelve cuántos bytes ocupa un tipo o variable.

Ejemplo:

-----**código**-----

```
printf("%zu\n", sizeof(int));  
printf("%zu\n", sizeof(char));  
printf("%zu\n", sizeof(double));
```

-----**código**-----

Normalmente:

sizeof(char) // 1 byte

sizeof(int) // 4 bytes normalmente

sizeof(double) // 8 bytes normalmente

Pero no debes asumirlo siempre. Por eso se usa sizeof.

Ejemplo básico

-----**código**-----

```
int x = 10;  
  
printf("%zu\n", sizeof(x));  
printf("%zu\n", sizeof(int));
```

-----**código**-----

Ambos suelen imprimir 4.

Caso importante con arrays

-----**código**-----

```
int nums[5];  
  
printf("%zu\n", sizeof(nums));
```

-----**código**-----

Si int ocupa 4 bytes, entonces:

sizeof(nums) == 20

Porque tienes 5 enteros:

5 * sizeof(int)

Pero cuidado:

-----**código**-----

```
void imprimir(int nums[]) {  
    printf("%zu\n", sizeof(nums));  
}
```

-----**código**-----

Aquí nums realmente es un puntero, no el array completo. Entonces sizeof(nums) devuelve el tamaño del puntero, no del array.

2. malloc

-----**código**-----

malloc significa memory allocation.

-----**código**-----

Sirve para pedir memoria dinámica.

Su sintaxis es:

-----**código**-----

```
puntero = malloc(numero_de_bytes);
```

-----**código**-----

Ejemplo:

-----**código**-----

```
int *p = malloc(sizeof(int));
```

-----**código**-----

Esto pide espacio para un entero.

Visualmente:

p ----> [int]

Ahora puedes usarlo así:

-----**código**-----

```
*p = 25;
printf("%d\n", *p);
```

-----**código**-----

Ejemplo completo

-----**código**-----

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = malloc(sizeof(int));

    if (p == NULL) {
        printf("Error reservando memoria\n");
        return 1;
    }

    *p = 42;

    printf("Valor: %d\n", *p);

    free(p);

    return 0;
}
```

-----**código**-----

¿Por qué comprobamos NULL?

Porque malloc puede fallar.

Si no hay memoria suficiente, devuelve:

NULL

Entonces esto sería peligroso:

-----**código**-----

```
int *p = malloc(sizeof(int));
*p = 5; // peligroso si p == NULL
```

-----**código**-----

Por eso se hace:

-----**código**-----

```
if (p == NULL) {
```

-----**código**-----

// error

-----**código**-----

```
}  
malloc para arrays
```

-----**código**-----

Si quieres un array dinámico de 5 enteros:

-----**código**-----

```
int *array = malloc(5 * sizeof(int));
```

-----**código**-----

Esto reserva espacio para:

-----**código**-----

```
array[0]  
array[1]  
array[2]  
array[3]  
array[4]
```

-----**código**-----

Puedes usarlo casi como un array normal:

-----**código**-----

```
array[0] = 10;  
array[1] = 20;  
array[2] = 30;
```

-----**código**-----

Ejemplo:

-----**código**-----

```
int *nums = malloc(5 * sizeof(int));  
  
if (nums == NULL) {  
    return 1;  
}  
  
for (int i = 0; i < 5; i++) {  
    nums[i] = i * 10;  
}  
  
for (int i = 0; i < 5; i++) {  
    printf("%d\n", nums[i]);  
}  
  
free(nums);
```

-----**código**-----

3. free

Toda memoria que pides con malloc, calloc, realloc o strdup debe liberarse con free.

Ejemplo:

-----**código**-----

```
int *p = malloc(sizeof(int));  
*p = 10;  
free(p);  
free(p) le dice al sistema:
```

-----**código**-----

Ya no necesito esta memoria.

Error grave: usar memoria después de free

-----**código**-----

```
int *p = malloc(sizeof(int));  
*p = 10;  
free(p);  
printf("%d\n", *p); // MAL
```

-----**código**-----

Después de free, p sigue guardando una dirección, pero esa memoria ya no te pertenece.

A esto se le llama:

use after free

Para evitarlo, a veces se hace:

-----**código**-----

```
free(p);  
p = NULL;
```

-----**código**-----

Así, si intentas usarlo, será más fácil detectar el error.

Error grave: hacer free dos veces

-----**código**-----

```
free(p);  
free(p); // MAL
```

-----**código**-----

Esto se llama:

double free

Puede romper el programa.

Mejor:

```
-----código-----  
    free(p);  
    p = NULL;
```

```
-----código-----
```

Porque:

```
-----código-----  
    free(NULL);
```

```
-----código-----
```

es seguro. No pasa nada.

4. calloc

```
-----código-----  
    calloc también reserva memoria dinámica, pero con una diferencia importante:  
    malloc
```

```
-----código-----
```

reserva memoria sin inicializar.

```
-----código-----  
    calloc
```

```
-----código-----
```

reserva memoria e inicializa todo a cero.

Sintaxis:

```
-----código-----  
    puntero = calloc(numero_elementos, tamaño_de_cada_elemento);
```

```
-----código-----
```

Ejemplo:

-----**código**-----

```
int *nums = calloc(5, sizeof(int));
```

-----**código**-----

Esto crea 5 enteros inicializados a 0.

Comparación

Con malloc:

-----**código**-----

```
int *nums = malloc(5 * sizeof(int));
```

-----**código**-----

Los valores iniciales son basura:

-----**código**-----

```
nums[0] // valor indeterminado  
nums[1] // valor indeterminado
```

-----**código**-----

Con calloc:

-----**código**-----

```
int *nums = calloc(5, sizeof(int));
```

-----**código**-----

Los valores son:

-----**código**-----

```
nums[0] == 0  
nums[1] == 0  
nums[2] == 0  
nums[3] == 0  
nums[4] == 0
```

-----**código**-----

Ejemplo

-----**código**-----

```
int *nums = calloc(5, sizeof(int));  
  
if (nums == NULL) {  
    return 1;  
}  
  
for (int i = 0; i < 5; i++) {
```



```
        printf("%d\n", nums[i]);
    }
    free(nums);
```

-----**código**-----

Imprime normalmente:

```
0
0
0
0
0
5. realloc
```

-----**código**-----

realloc sirve para cambiar el tamaño de una zona de memoria ya reservada.

-----**código**-----

Ejemplo:

-----**código**-----

```
int *nums = malloc(3 * sizeof(int));
```

-----**código**-----

Tienes espacio para 3 enteros.

Después quieres espacio para 6:

-----**código**-----

```
nums = realloc(nums, 6 * sizeof(int));
```

-----**código**-----

Problema importante

No deberías hacer directamente esto:

-----**código**-----

```
nums = realloc(nums, nuevo_tamaño);
```

-----**código**-----

Porque si realloc falla, devuelve NULL, y pierdes la dirección original.

Mejor:

-----**código**-----

```
int *temp = realloc(nums, nuevo_tamaño);
if (temp == NULL) {
```

-----**código**-----

// nums sigue siendo válido

-----**código**-----

```
    } else {
        nums = temp;
    }
```

-----**código**-----

Ejemplo correcto

-----**código**-----

```
int *nums = malloc(3 * sizeof(int));

if (nums == NULL) {
    return 1;
}

nums[0] = 10;
nums[1] = 20;
nums[2] = 30;

int *temp = realloc(nums, 6 * sizeof(int));

if (temp == NULL) {
    free(nums);
    return 1;
}

nums = temp;

nums[3] = 40;
nums[4] = 50;
nums[5] = 60;

for (int i = 0; i < 6; i++) {
    printf("%d\n", nums[i]);
}

free(nums);
```

-----**código**-----

¿Qué hace internamente realloc?

Puede pasar una de dos cosas:

Caso 1: puede ampliar la memoria en el mismo sitio

-----**código**-----

```
nums ----> [10][20][30][ ][ ][ ]
```

-----**código**-----

Caso 2: no puede ampliarla ahí

Entonces busca otra zona más grande, copia los datos antiguos y libera la anterior.

antes:

```
-----código-----  
    nums ----> [10][20][30]
```

```
-----código-----
```

después:

```
-----código-----  
    nums ----> [10][20][30][ ][ ][ ]
```

```
-----código-----
```

Pero quizá en otra dirección.

Por eso no debes guardar punteros internos antiguos.

Ejemplo peligroso:

```
-----código-----  
    int *nums = malloc(3 * sizeof(int));  
    int *p = &nums[1];  
  
    int *temp = realloc(nums, 100 * sizeof(int));  
    nums = temp;  
  
    *p = 50; // peligroso, p puede apuntar a memoria antigua
```

```
-----código-----
```

6. strdup

strdup duplica una cadena reservando memoria dinámica.

Ejemplo:

```
-----código-----  
    char *nombre = strdup("Diego");
```

```
-----código-----
```

Eso crea una copia dinámica de "Diego".

Equivale aproximadamente a:

```
-----código-----  
    char *nombre = malloc(strlen("Diego") + 1);
```

```
strcpy(nombre, "Diego");
```

-----**código**-----

El +1 es por el carácter final '\0'.

Ejemplo

-----**código**-----

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char *nombre = strdup("Diego");

    if (nombre == NULL) {
        return 1;
    }

    printf("%s\n", nombre);

    free(nombre);

    return 0;
}
```

-----**código**-----

Detalle importante

strdup reserva memoria, así que también se libera con:

-----**código**-----

```
free(nombre);
```

-----**código**-----

Aunque uses cadenas, sigue siendo memoria dinámica.

7. Bloques de memoria

Ahora pasamos al segundo subapartado:

-----**código**-----

```
memcpy
memset
memcmp
```

-----**código**-----

Estas funciones trabajan con memoria en bruto.

No trabajan necesariamente con int, char, struct, etc. Trabajan con bloques de bytes.

Están en:

-----**código**-----

```
#include <string.h>
```

-----**código**-----

8. memset

-----**código**-----

memset rellena una zona de memoria con un valor byte a byte.

-----**código**-----

Sintaxis:

-----**código**-----

```
memset(direccion, valor, cantidad_de_bytes);
```

-----**código**-----

Ejemplo:

-----**código**-----

```
int nums[5];  
memset(nums, 0, 5 * sizeof(int));
```

-----**código**-----

Esto pone todos los bytes del array a 0.

Para enteros, poner todos los bytes a cero normalmente equivale a enteros con valor 0.

Ejemplo con chars

-----**código**-----

```
char texto[10];  
memset(texto, 'A', 9);  
texto[9] = '\0';  
printf("%s\n", texto);
```

-----**código**-----

Resultado:

AAAAAAAAA

Cuidado con memset en enteros

Esto parece lógico:

-----**código**-----

```
int nums[5];  
memset(nums, 1, 5 * sizeof(int));
```

-----**código**-----

Pero no pone cada entero a 1.

Pone cada byte a 1.

Si un int ocupa 4 bytes, cada entero queda así:

00000001 00000001 00000001 00000001

Eso no es el número 1. Es otro valor.

Para inicializar enteros a 1, usa un bucle:

-----**código**-----

```
for (int i = 0; i < 5; i++) {  
    nums[i] = 1;  
}
```

-----**código**-----

9. memcpy

-----**código**-----

memcpy copia bytes de una zona de memoria a otra.

-----**código**-----

Sintaxis:

-----**código**-----

```
memcpy(destino, origen, cantidad_de_bytes);
```

-----**código**-----

Ejemplo:

-----**código**-----

```
int origen[3] = {10, 20, 30};  
int destino[3];  
  
memcpy(destino, origen, 3 * sizeof(int));
```

-----**código**-----

Ahora destino contiene:

10, 20, 30

Ejemplo con estructuras

-----**código**-----

```
typedef struct {
    int id;
    char nombre[20];
} Persona;

Persona p1 = {1, "Ana"};
Persona p2;

memcpy(&p2, &p1, sizeof(Persona));
```

-----**código**-----

Ahora p2 es una copia byte a byte de p1.

Cuidado con memoria solapada

-----**código**-----

```
memcpy no debe usarse si origen y destino se solapan.
```

-----**código**-----

Ejemplo peligroso:

-----**código**-----

```
char texto[] = "abcdef";

memcpy(texto + 1, texto, 3);
```

-----**código**-----

Aquí origen y destino están en el mismo array y se pisan.

Para memoria solapada existe:

memmove

Aunque no aparece en tu apartado, es bueno conocerlo.

10. memcmp

-----**código**-----

```
memcmp compara dos bloques de memoria byte a byte.
```

-----**código**-----

Sintaxis:

-----**código**-----

```
memcmp(bloque1, bloque2, cantidad_de_bytes);
```

-----**código**-----

Devuelve:

0 si son iguales

<0 si el primer bloque es menor

>0 si el primer bloque es mayor

Ejemplo:

-----**código**-----

```
int a[3] = {1, 2, 3};
int b[3] = {1, 2, 3};

if (memcmp(a, b, 3 * sizeof(int)) == 0) {
    printf("Son iguales\n");
}
```

-----**código**-----

Cuidado con estructuras

Esto puede ser peligroso:

-----**código**-----

```
typedef struct {
    char c;
    int x;
} Dato;
```

-----**código**-----

Una estructura puede tener bytes de relleno internos, llamados padding.

Entonces dos estructuras con los mismos campos podrían no tener exactamente los mismos bytes.

Por eso, para estructuras, muchas veces es mejor comparar campo a campo:

-----**código**-----

```
if (d1.c == d2.c && d1.x == d2.x) {
    printf("Iguales\n");
}
```

-----**código**-----

Código usando todo el apartado 10

Este programa usa:

-----**código**-----

```
malloc
calloc
```


-----**código**-----

sizeof

-----**código**-----

realloc

-----**código**-----

strdup

free

-----**código**-----

```
memset
memcpy
memcmp
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char *nombre;
    int edad;
} Persona;

int main() {
    int n = 3;
```

-----**código**-----

// malloc + sizeof

-----**código**-----

```
int *numeros = malloc(n * sizeof(int));

if (numeros == NULL) {
    printf("Error con malloc\n");
    return 1;
}

for (int i = 0; i < n; i++) {
    numeros[i] = (i + 1) * 10;
}

printf("Array creado con malloc:\n");
for (int i = 0; i < n; i++) {
    printf("%d ", numeros[i]);
}
printf("\n\n");
```

-----**código**-----

// calloc

-----**código**-----

```
int *ceros = calloc(n, sizeof(int));

if (ceros == NULL) {
    printf("Error con calloc\n");
}
```

```

        free(numeros);
        return 1;
    }

    printf("Array creado con calloc:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", ceros[i]);
    }
    printf("\n\n");

```

-----**código**-----

// realloc

-----**código**-----

```

    int nuevoTam = 5;
    int *temp = realloc(numeros, nuevoTam * sizeof(int));

    if (temp == NULL) {
        printf("Error con realloc\n");
        free(numeros);
        free(ceros);
        return 1;
    }

    numeros = temp;

    numeros[3] = 40;
    numeros[4] = 50;

    printf("Array después de realloc:\n");
    for (int i = 0; i < nuevoTam; i++) {
        printf("%d ", numeros[i]);
    }
    printf("\n\n");

```

-----**código**-----

// strdup

-----**código**-----

```

    Persona p1;
    p1.id = 1;
    p1.nombre = strdup("Diego");
    p1.edad = 20;

    if (p1.nombre == NULL) {
        printf("Error con strdup\n");
        free(numeros);
        free(ceros);
        return 1;
    }

    printf("Persona p1:\n");
    printf("ID: %d\n", p1.id);
    printf("Nombre: %s\n", p1.nombre);
    printf("Edad: %d\n\n", p1.edad);

```

-----**código**-----

// memcpy

-----**código**-----

```

    Persona p2;

```

```
memcpy(&p2, &p1, sizeof(Persona));
```

-----**código**-----

```
/*
```

OJO:

Esto copia también el puntero nombre.

Es decir, p2.nombre apunta al mismo sitio que p1.nombre.

Por eso NO haremos free(p2.nombre),
porque sería un double free.

```
*/
```

-----**código**-----

```
printf("Persona p2 copiada con memcpy:\n");  
printf("ID: %d\n", p2.id);  
printf("Nombre: %s\n", p2.nombre);  
printf("Edad: %d\n\n", p2.edad);
```

-----**código**-----

```
// memcmp
```

-----**código**-----

```
if (memcmp(numeros, ceros, n * sizeof(int)) == 0) {  
    printf("Los primeros elementos de numeros y ceros son iguales\n");  
} else {  
    printf("Los primeros elementos de numeros y ceros son diferentes\n");  
}
```

-----**código**-----

```
// memset
```

-----**código**-----

```
memset(ceros, 0, n * sizeof(int));  
  
printf("\nArray ceros después de memset:\n");  
for (int i = 0; i < n; i++) {  
    printf("%d ", ceros[i]);  
}  
printf("\n");
```

-----**código**-----

```
// free
```

-----**código**-----

```
free(numeros);  
free(ceros);  
free(p1.nombre);  
  
numeros = NULL;  
ceros = NULL;  
p1.nombre = NULL;  
p2.nombre = NULL;  
return 0;
```

```
}
```

-----***código***-----

Compilación:

```
gcc programa.c -o programa
```

Ejecución:

```
./programa
```

En Windows:

```
programa.exe
```